SYSTEM ARCHITECTURE DOCUMENT

1. Project Overview

Project: AI-Powered Integrated Bid Compliance Verification Platform for GeM Procurement
SIH Problem Statement: 26100

The system is a secure, AI-assisted procurement compliance platform that analyzes tender requirements, processes bidder documents, verifies extracted information, evaluates compliance, routes uncertain cases to human reviewers, and generates an auditable final compliance report.

The architecture must be:

- Modular
- Secure
- Explainable
- Auditable
- Scalable
- Fault tolerant
- Human-in-the-loop
- API-first

---

2. Architectural Principle

The core decision pipeline is:

```
Tender
  ↓
Requirement Extraction
  ↓
Structured Requirements
  ↓
Bid Submission
  ↓
Document Processing
  ↓
OCR / Parsing
  ↓
Entity & Field Extraction
  ↓
Evidence Store
  ↓
Rule Engine
  +
AI Analysis
```

```
  +
Verification
  ↓
Compliance Decision
  ↓
Risk / Confidence
  ↓
Human Review
  ↓
Final Decision
  ↓
Report + Audit Trail
```

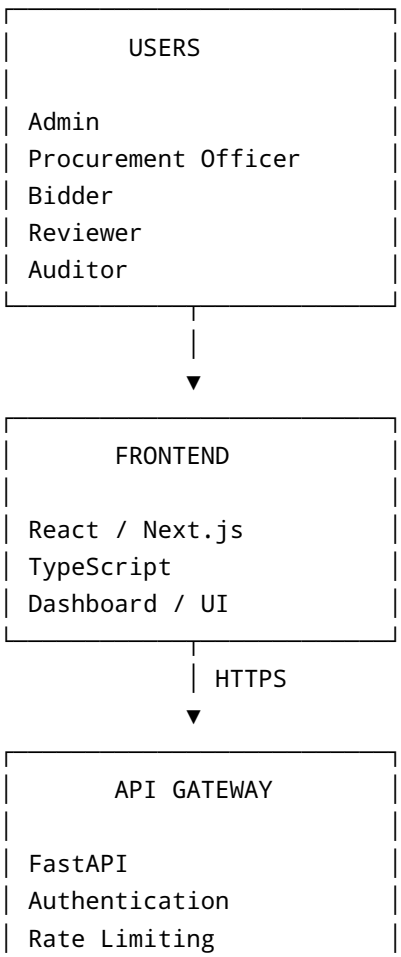
The AI must NOT independently become the final authority for critical procurement decisions.

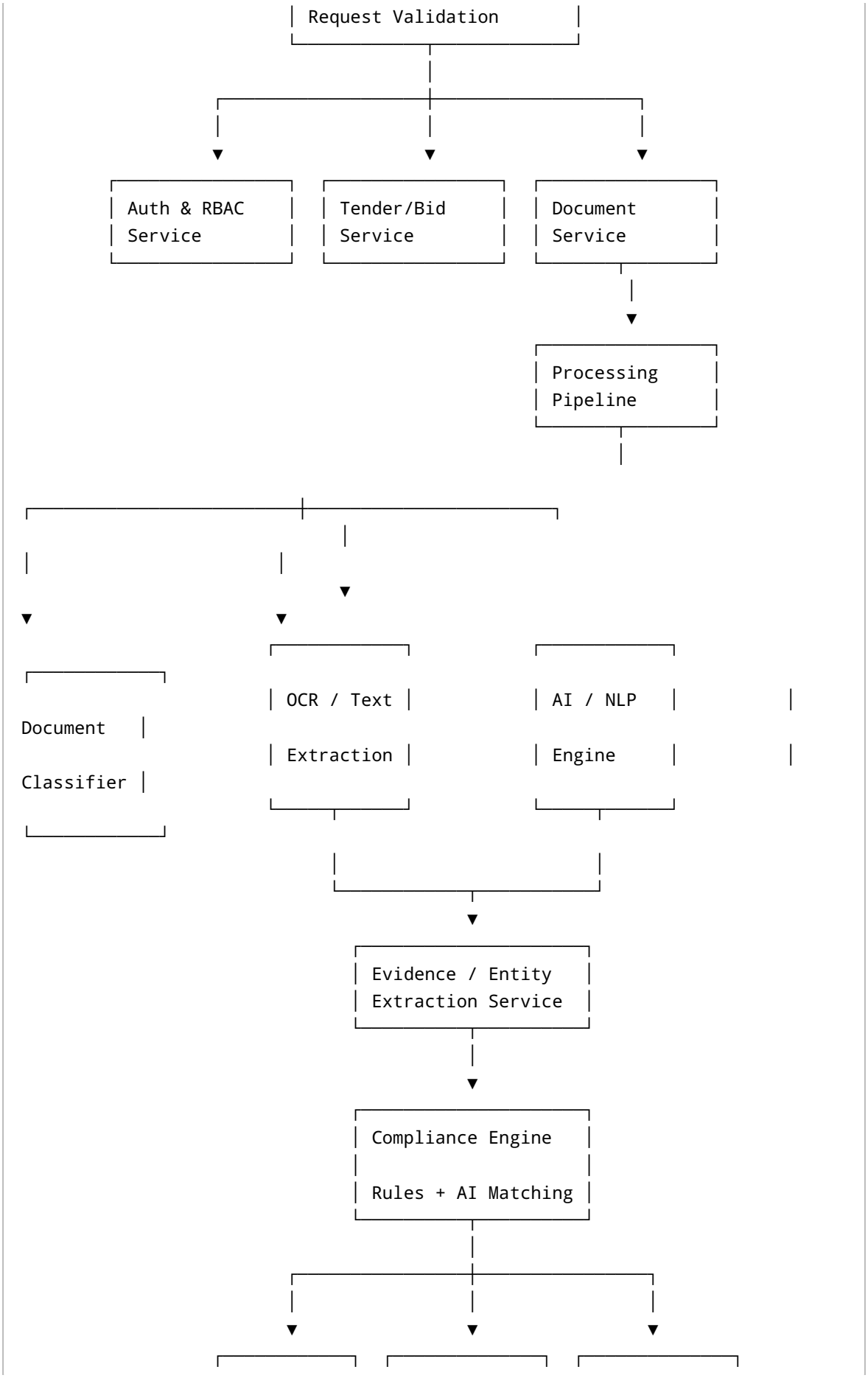
The architecture combines:

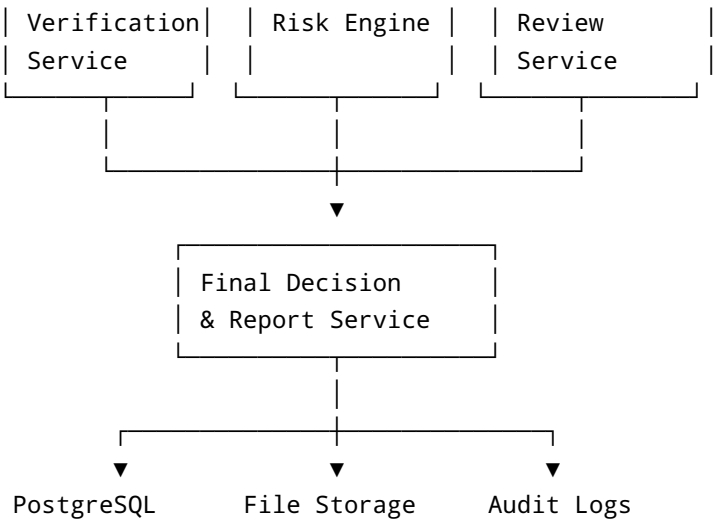
AI + Rules + Verification + Human Review

---

3. High-Level Architecture




Request Validation
Auth & RBAC Service
Tender/Bid Service
Document Service
Processing Pipeline
Document Classifier
OCR / Text Extraction
AI / NLP Engine
Evidence / Entity Extraction Service
Compliance Engine
Rules + AI Matching



---

4. Architectural Style

Use a **modular service-oriented monolith for the initial MVP**, with clearly separated internal modules.

Do NOT immediately create dozens of independent microservices.

Recommended initial structure:

```
Frontend
    ↓
FastAPI Application
    ↓
Modular Backend Services
    ↓
PostgreSQL + Object Storage
    ↓
Background Workers
```

The modules should have clear interfaces so they can later be extracted into independent services if scale requires it.

---

5. Frontend Architecture

Technology

- React / Next.js
- TypeScript
- Tailwind CSS

- Component-based architecture

Responsibilities

Frontend handles:

- Authentication UI
- Role-specific dashboards
- Tender management
- Bid submission
- Document upload
- Processing status
- Compliance visualization
- Review interface
- Reports
- Audit interface

Frontend MUST NOT implement security decisions.

For example, hiding an "Admin" button does NOT provide authorization.

The backend must independently verify permissions.

---

6. Backend Architecture

Technology

- Python
- FastAPI
- SQLAlchemy
- Pydantic
- PostgreSQL

Recommended structure:

```
backend/
│
├── app/
│   │
│   ├── main.py
│   │
│   ├── api/
│   │   └── v1/
│   │       ├── auth.py
│   │       ├── users.py
│   │       ├── tenders.py
│   │       ├── bids.py
│   │       ├── documents.py
```

```
│   │       ├── compliance.py
│   │       ├── verification.py
│   │       ├── reviews.py
│   │       ├── reports.py
│   │       └── audit.py
│   │
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── database.py
│   │   └── logging.py
│   │
│   ├── models/
│   │   ├── user.py
│   │   ├── tender.py
│   │   ├── bid.py
│   │   ├── document.py
│   │   ├── requirement.py
│   │   ├── compliance.py
│   │   ├── verification.py
│   │   ├── review.py
│   │   └── audit.py
│   │
│   ├── schemas/
│   │
│   ├── services/
│   │   ├── tender_service.py
│   │   ├── bid_service.py
│   │   ├── document_service.py
│   │   ├── report_service.py
│   │   └── notification_service.py
│   │
│   ├── ai/
│   │   ├── requirement_extractor.py
│   │   ├── document_classifier.py
│   │   ├── entity_extractor.py
│   │   ├── compliance_analyzer.py
│   │   └── provider.py
│   │
│   ├── ocr/
│   │   ├── processor.py
│   │   └── provider.py
│   │
│   ├── compliance/
│   │   ├── engine.py
│   │   ├── rules.py
│   │   ├── matcher.py
│   │   └── evaluator.py
│   │
│   ├── verification/
```

```
│   │   ├── service.py
│   │   ├── base.py
│   │   ├── mock_provider.py
│   │   └── providers/
│   │
│   ├── review/
│   │
│   ├── audit/
│   │
│   └── repositories/
│
├── tests/
├── migrations/
└── requirements.txt
```

---

7. Authentication Architecture

Authentication flow:

```
User
 ↓
Login
 ↓
FastAPI Authentication Endpoint
 ↓
Validate Credentials
 ↓
Password Hash Verification
 ↓
Generate Access Token
 ↓
Frontend Stores Token Securely
 ↓
Authenticated API Request
 ↓
Authentication Middleware
 ↓
User Identity
 ↓
RBAC / Resource Authorization
 ↓
Endpoint
```

Use strong password hashing.

Never store plaintext passwords.

Secrets must be loaded from environment/secret management.

---

8. Authorization Architecture

Authorization must happen at two levels.

Role Authorization

Example:

```
Admin → user management
Officer → tender management
Bidder → own bids
Reviewer → assigned reviews
Auditor → audit read access
```

Resource Authorization

Example:

```
Bidder A
   ↓
GET /bids/BID_A
   ↓
Allowed

Bidder A
   ↓
GET /bids/BID_B
   ↓
Forbidden
```

Never rely solely on user-supplied IDs.

The backend must check ownership/relationship.

---

9. Tender Processing Architecture

```
Officer
  ↓
Create Tender
  ↓
```

```
Upload Tender Document
  ↓
Document Validation
  ↓
Secure Storage
  ↓
Create Processing Job
  ↓
Text Extraction / OCR
  ↓
Requirement Extraction
  ↓
Requirement Normalization
  ↓
Requirement Validation
  ↓
Store Structured Requirements
```

---

10. Document Processing Architecture

Documents pass through:

```
Upload
 ↓
MIME/File Validation
 ↓
Size Validation
 ↓
Security Validation
 ↓
Hash Generation
 ↓
Secure Storage
 ↓
Document Classification
 ↓
Text Extraction
 ↓
OCR if necessary
 ↓
Field/Entity Extraction
 ↓
Normalization
 ↓
Evidence Creation
```

The original file must remain available according to retention/access policies.

---

11. OCR Architecture

Use an OCR abstraction:

```
OCRService
    │
    ├── LocalOCRProvider
    │
    ├── CloudOCRProvider
    │
    └── FutureOCRProvider
```

Application code should depend on the interface rather than a specific OCR vendor.

Example conceptual interface:

```
extract_text(document)
→ OCRResult
```

OCRResult should contain:

- Text
- Pages
- Confidence
- Processing status
- Error information

---

12. AI Architecture

Use an AI provider abstraction.

```
AIService
   │
   ├── Requirement Extraction
   ├── Document Classification
   ├── Entity Extraction
   ├── Evidence Matching
   └── Compliance Assistance
```

Provider abstraction:

```
AIProvider
   │
   ├── Provider A
   ├── Provider B
   └── Local/Future Provider
```

The rest of the application must not depend directly on one AI vendor.

---

13. AI Processing Pipeline

Requirement Extraction

```
Tender Text
   ↓
Chunking if required
   ↓
AI Extraction
   ↓
Structured JSON
   ↓
Schema Validation
   ↓
Requirement Normalization
   ↓
Database
```

Document Extraction

```
Document
   ↓
OCR/Text
   ↓
Classification
   ↓
AI Entity Extraction
   ↓
Schema Validation
   ↓
Evidence Store
```

AI output must be validated before entering the database.

AI-generated values should never automatically be treated as verified facts.

---

14. Evidence Architecture

Evidence is a first-class object in the system.

An evidence record should connect:

```
Requirement
     │
     ▼
Evidence
     │
     ├── Document
     ├── Page
     ├── Extracted text
     ├── Extracted field
     ├── Confidence
     └── Source
```

Example:

```
Requirement:
Minimum turnover ₹50 lakh

Evidence:
Document = FinancialStatement.pdf
Page = 4
Extracted value = ₹72 lakh
Confidence = 0.94
```

This enables explainable decisions.

---

15. Compliance Engine Architecture

The compliance engine should consist of multiple layers.

```
Requirement
     +
Evidence
     ↓
Evidence Matcher
     ↓
Rule Engine
     ↓
AI Semantic Analysis where necessary
```

```
     ↓
Verification Result
     ↓
Decision Aggregator
     ↓
Compliance Result
```

Possible results:

```
COMPLIANT
NON_COMPLIANT
NEEDS_REVIEW
NOT_APPLICABLE
```

---

16. Rule Engine

Rules should be deterministic whenever possible.

Examples:

```
IF turnover >= required_turnover
THEN COMPLIANT
```

```
IF certificate_expiry < relevant_date
THEN NON_COMPLIANT
```

```
IF mandatory_document_missing
THEN NON_COMPLIANT
```

```
IF evidence_confidence < threshold
THEN NEEDS_REVIEW
```

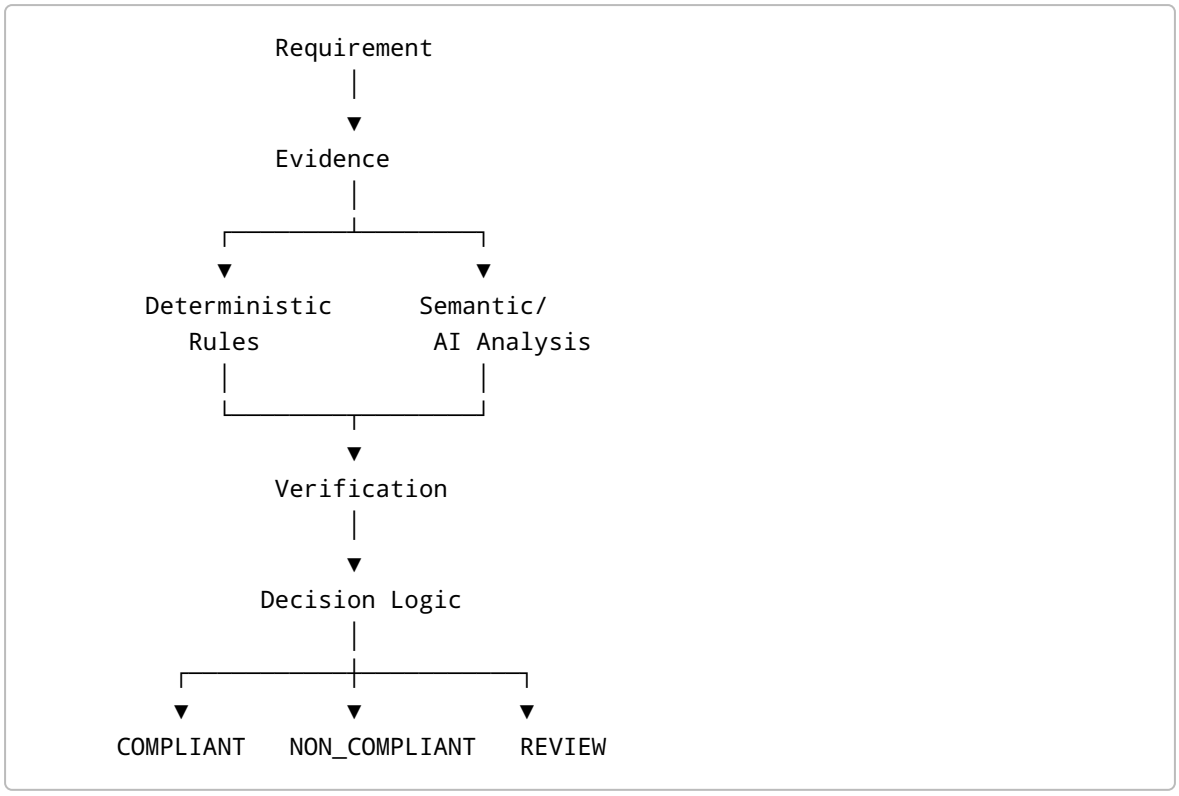
Rules should be modular and testable.

Avoid embedding all rules directly inside API endpoints.

---

17. AI + Rule Decision Architecture

Decision priority:



The system should favor deterministic rules when a requirement can be objectively evaluated.

---

18. Cross-Document Verification

Architecture:

```
Document A
   ↓
Entity Extraction
   ↓
Document B
   ↓
Entity Extraction
   ↓
Normalization
   ↓
Entity Matching
   ↓
Consistency Analysis
   ↓
Flag / Pass
```

Examples:

```
GST Name
   ↕
PAN Name
   ↕
Registration Name
```

Potential mismatch:

```
ABC Technologies Pvt Ltd
vs
ABC Technology Pvt Ltd
```

This should create a review flag, not automatically classify the bidder as fraudulent.

---

19. Verification Architecture

Use an adapter pattern:

```
VerificationService
        │
        ├── GSTVerificationProvider
        ├── CertificateVerificationProvider
        ├── OrganizationVerificationProvider
        └── MockVerificationProvider
```

Each provider follows a common interface.

```
verify(entity)
    ↓
VerificationResult
```

VerificationResult:

```
status
source
reference
timestamp
result
evidence
error
```

---

20. Mock Verification

For SIH development/demo, use controlled synthetic verification data.

Example:

```
Mock GST Provider
        ↓
Lookup GSTIN
        ↓
Synthetic Response
        ↓
VERIFIED
```

The UI must clearly display:

MOCK VERIFICATION

It must never be represented as an actual government verification.

---

21. Background Job Architecture

Long-running operations should not block normal HTTP requests.

Use background workers for:

- OCR
- AI processing
- Requirement extraction
- Document classification
- Compliance processing
- Verification
- Report generation

Architecture:

```
API
 ↓
Create Job
 ↓
Queue
 ↓
Worker
 ↓
Process
 ↓
```

```
Update Database
 ↓
Notify Frontend
```

For MVP, FastAPI background tasks may be used where sufficient.

For scalable deployment, use a dedicated queue/worker system.

---

22. Database Architecture

Use PostgreSQL as the primary relational database.

Core relationships:

```
User
 │
 ├── Bids
 └── Reviews

Tender
 │
 ├── Tender Documents
 ├── Requirements
 └── Bids

Bid
 │
 └── Bid Documents
        │
        └── Extractions
               │
               └── Evidence

Requirement
 │
 └── Compliance Result
        │
        ├── Evidence
        ├── Verification
        └── Review

All important entities
 │
 └── Audit Logs
```

---

23. Suggested Database Entities

```
users
roles
permissions
user_roles

tenders
tender_documents
tender_requirements

bids
bid_documents

document_extractions
extracted_entities
evidence

compliance_results
compliance_rules

verification_records

review_tasks
review_decisions

reports
notifications

audit_logs
```

Use foreign keys and indexes.

Use transactions for important state changes.

---

24. File Storage Architecture

Do not store large document binaries directly in PostgreSQL unless there is a specific reason.

Use object/file storage.

```
Application
     ↓
Secure File Service
     ↓
Object Storage
```

```
      │
      ├── Tender Documents
      ├── Bid Documents
      ├── Generated Reports
      └── Processing Artifacts
```

Database stores metadata and secure storage identifiers.

---

25. Secure File Access

Files must never be exposed through unrestricted public URLs.

Preferred flow:

```
User
 ↓
Request Document
 ↓
Backend Authentication
 ↓
Authorization
 ↓
Check Bid/Tender Access
 ↓
Generate controlled access
 ↓
Return file
```

A user changing a document ID must not expose another user's document.

---

26. Audit Architecture

Important actions generate audit events.

```
User Action
     ↓
Application Service
     ↓
Business Operation
     ↓
Audit Event
     ↓
Audit Log
```

Example:

```
Actor: reviewer_123
Action: COMPLIANCE_DECISION_CHANGED
Resource: compliance_456
Previous: NEEDS_REVIEW
New: COMPLIANT
Timestamp: ...
Reason: ...
```

Audit logging must not expose unnecessary sensitive document content.

---

27. Reporting Architecture

```
Bid
 ↓
Compliance Results
 ↓
Verification Results
 ↓
Reviewer Decisions
 ↓
Audit References
 ↓
Report Service
 ↓
PDF Report
```

Reports should be generated asynchronously if processing is expensive.

---

28. API Layer

Use versioned REST APIs.

```
/api/v1/auth
/api/v1/users
/api/v1/tenders
/api/v1/bids
/api/v1/documents
/api/v1/compliance
/api/v1/verification
/api/v1/reviews
```

```
/api/v1/reports
/api/v1/audit
```

Use:

- Pydantic schemas
- Consistent errors
- HTTP status codes
- Authentication dependencies
- Authorization dependencies
- Request validation

---

29. API Request Flow

```
HTTP Request
     ↓
Middleware
     ↓
Authentication
     ↓
Authorization
     ↓
Validation
     ↓
API Router
     ↓
Service Layer
     ↓
Repository / External Service
     ↓
Database
     ↓
Response Schema
     ↓
HTTP Response
```

Routes should remain thin.

Business logic belongs in services.

---

30. Error Handling Architecture

Use centralized error handling.

Categories:

```
AuthenticationError
AuthorizationError
ValidationError
NotFoundError
FileProcessingError
OCRProcessingError
AIProcessingError
VerificationError
DatabaseError
```

Return safe messages.

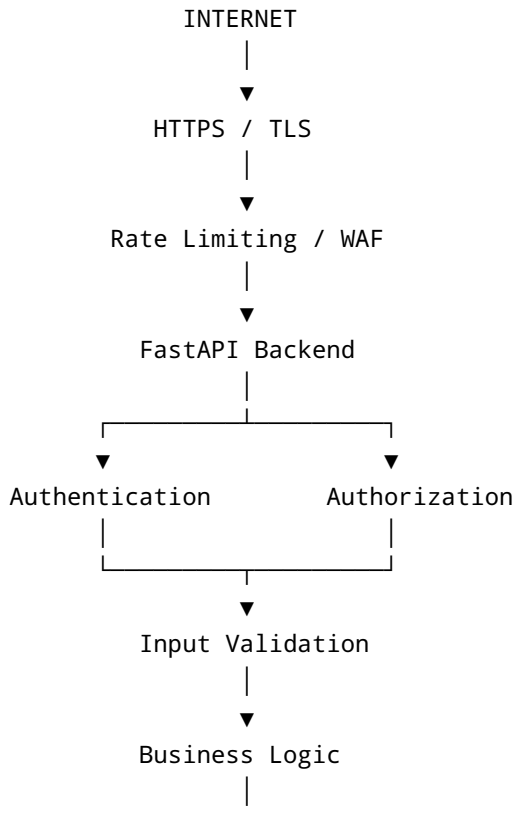
Never return:

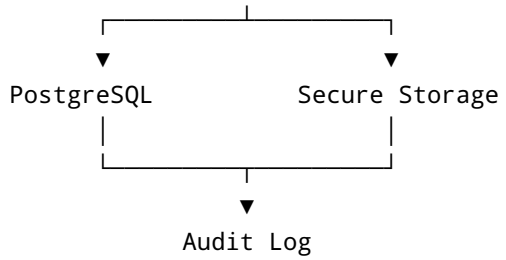
- Stack traces
- Database credentials
- API keys
- Internal paths
- Sensitive infrastructure details

---

31. Security Architecture

Security layers:





---

32. Security Controls

Implement:

- Strong password hashing
- Token expiration
- Secure authentication
- Backend RBAC
- Object-level authorization
- Input validation
- SQLAlchemy parameterization
- Secure file upload
- MIME validation
- File size limits
- Path traversal protection
- Secret management
- HTTPS
- Encryption in transit
- Encryption at rest where supported
- Rate limiting
- Audit logs
- Secure error handling

---

33. Data Security

Sensitive data should be protected throughout its lifecycle.

```
Upload
 ↓
Encrypted Transport
 ↓
Secure Validation
 ↓
Encrypted Storage
 ↓
Authorized Processing
 ↓
```

```
Controlled Access
 ↓
Audit
 ↓
Retention/Deletion Policy
```

Do not log full sensitive documents or secrets.

---

34. Observability

Implement structured application logging.

Track:

- Request ID
- User ID where appropriate
- Operation
- Processing status
- Error category
- Processing duration

Do not log:

- Passwords
- API keys
- Authentication tokens
- Full sensitive documents
- Unnecessary personal information

---

35. Monitoring

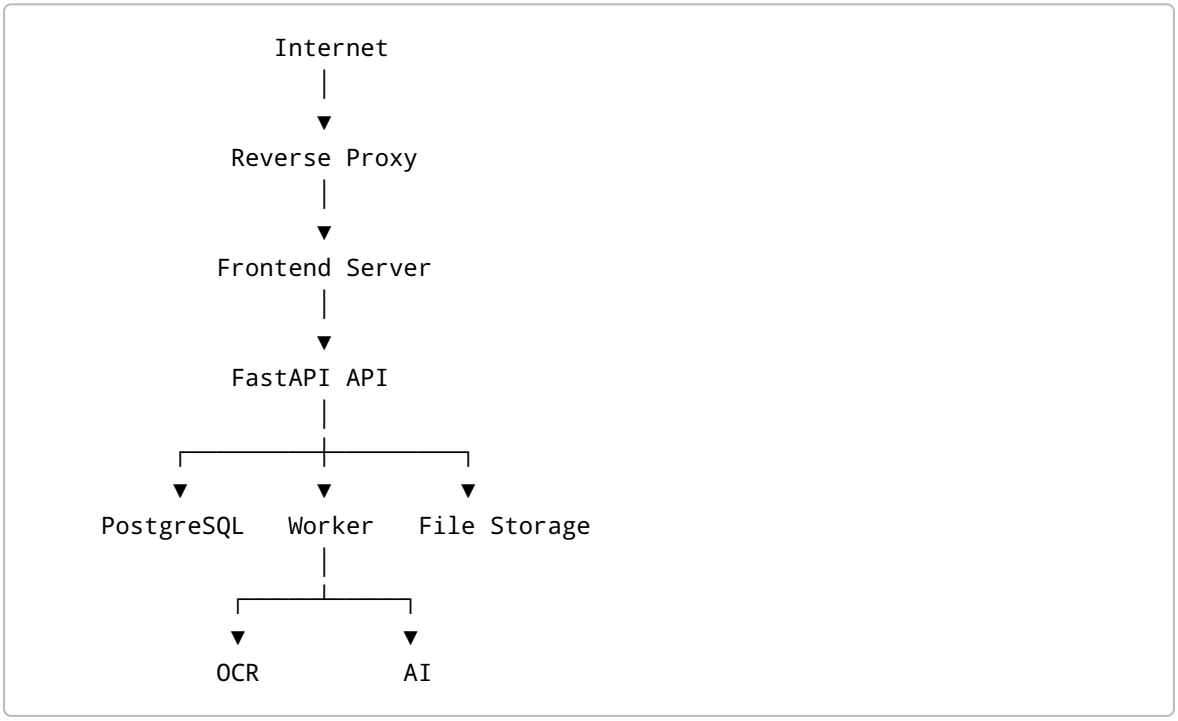
Monitor:

- API latency
- Error rate
- Database performance
- OCR failures
- AI failures
- Queue size
- Worker failures
- Verification failures
- Document-processing duration

---

36. Deployment Architecture

Recommended deployment:



Use Docker for consistent development and deployment.

---

37. Development Environment

Recommended:

```
project/
├── frontend/
├── backend/
├── worker/
├── docker-compose.yml
├── .env.example
├── README.md
└── docs/
```

Never commit `.env` containing real secrets.

Provide `.env.example` with placeholder values.

---

38. Configuration Architecture

Centralize configuration.

Example categories:

```
DATABASE_URL
JWT configuration
FILE_STORAGE configuration
AI provider configuration
OCR configuration
VERIFICATION configuration
CORS configuration
APPLICATION configuration
```

All secrets must come from environment variables or a proper secret manager.

---

39. State Management

Tender states:

```
DRAFT
PUBLISHED
CLOSED
UNDER_EVALUATION
COMPLETED
CANCELLED
```

Bid states:

```
DRAFT
SUBMITTED
PROCESSING
UNDER_REVIEW
COMPLIANT
NON_COMPLIANT
FINALIZED
```

Document states:

```
UPLOADED
VALIDATING
PROCESSING
PROCESSED
```

```
FAILED
REVIEW_REQUIRED
```

Review states:

```
PENDING
IN_PROGRESS
APPROVED
REJECTED
CLARIFICATION_REQUIRED
COMPLETED
```

State transitions should be controlled by backend business rules.

---

40. End-to-End Architecture Flow

Phase 1 — Tender

```
Officer
 ↓
Create Tender
 ↓
Upload Tender PDF
 ↓
Validate
 ↓
Store
 ↓
OCR/Text Extraction
 ↓
AI Requirement Extraction
 ↓
Normalize Requirements
 ↓
Store Requirements
```

Phase 2 — Bid

```
Bidder
 ↓
Select Tender
 ↓
Submit Bid
 ↓
```

```
Upload Documents
 ↓
Validation
 ↓
Secure Storage
```

Phase 3 — AI Processing

```
Documents
 ↓
OCR
 ↓
Classification
 ↓
Entity Extraction
 ↓
Normalization
 ↓
Evidence Creation
```

Phase 4 — Compliance

```
Requirements
      +
Evidence
      ↓
Matching
      ↓
Rule Engine
      +
AI Analysis
      +
Verification
      ↓
Compliance Results
```

Phase 5 — Human Review

```
Low Confidence / Conflict / Missing Evidence
                      ↓
                Review Task
                      ↓
                  Reviewer
                      ↓
             Final Decision
```

Phase 6 — Reporting

```
Final Decision
     +
Evidence
     +
Verification
     +
Review History
     ↓
Compliance Report
     ↓
Audit Trail
```

---

41. Failure Handling

If AI fails:

```
AI Failure
 ↓
Store failure status
 ↓
Retry
 ↓
If repeated failure
 ↓
Manual Review
```

If OCR fails:

```
OCR Failure
 ↓
Mark document FAILED
 ↓
Retry / Alternative OCR
 ↓
Manual Review if required
```

If verification service is unavailable:

```
Verification Unavailable
 ↓
Do NOT mark VERIFIED
 ↓
```

```
Set VERIFICATION_UNAVAILABLE
 ↓
Route to manual review if required
```

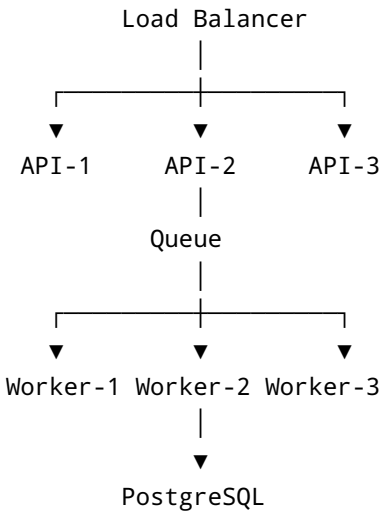
---

42. Scalability Strategy

Initial deployment:

```
1 API
1 Worker
1 PostgreSQL
1 File Storage
```

Future scaling:



Stateless API servers should be preferred.

---

43. Testing Architecture

Testing layers:

```
Unit Tests
    ↓
Service Tests
    ↓
Integration Tests
    ↓
```

```
API Tests
    ↓
Security Tests
    ↓
End-to-End Tests
```

Critical security tests must include:

- IDOR prevention
- Role escalation prevention
- Unauthorized document access
- Invalid authentication
- Malicious file upload
- Input injection
- Token misuse

---

44. AI Reliability Requirements

The AI layer must:

- Use structured output
- Validate output schema
- Preserve source evidence
- Provide confidence where appropriate
- Avoid unsupported claims
- Detect missing evidence
- Support fallback to human review

Never allow an AI hallucination to become an unsupported compliance fact.

---

45. Architecture Decision Summary

Area	Decision
Frontend	React / Next.js
Backend	FastAPI
Language	Python + TypeScript
Database	PostgreSQL
ORM	SQLAlchemy
Validation	Pydantic
AI	Provider abstraction
OCR	Provider abstraction

Area	Decision
Storage	Secure object/file storage
API	REST
Architecture	Modular service-oriented monolith
Background jobs	Worker/queue architecture
Authentication	Token/session based
Authorization	RBAC + resource-level authorization
Reporting	PDF generation
Deployment	Docker-ready
Auditing	Dedicated audit subsystem

---

46. Golden Architecture Rule

The implementation must follow this separation:

```
FRONTEND
    ↓
API
    ↓
AUTH + RBAC
    ↓
SERVICE LAYER
    ↓
DOMAIN LOGIC
    ├── Tender
    ├── Bid
    ├── Document
    ├── AI
    ├── OCR
    ├── Compliance
    ├── Verification
    ├── Review
    └── Reporting
    ↓
DATABASE / STORAGE / EXTERNAL PROVIDERS
```

Do not mix:

- UI logic with business logic
- API routes with complex compliance rules
- AI provider code with domain logic

- File storage logic with authentication logic
- Database queries throughout unrelated modules

The architecture must remain modular so individual components can be replaced or scaled without rewriting the entire platform.

---

47. Final Architecture Objective

The finished system must provide this complete chain:

Tender → Requirements → Bid → Documents → OCR → AI Extraction → Evidence → Rule Engine → AI Analysis → Verification → Risk/Confidence → Human Review → Final Compliance Decision → Report → Audit Trail

The architecture must ensure that every important decision is:

Secure + Evidence-Based + Explainable + Reviewable + Auditable.